Deploy Blindado

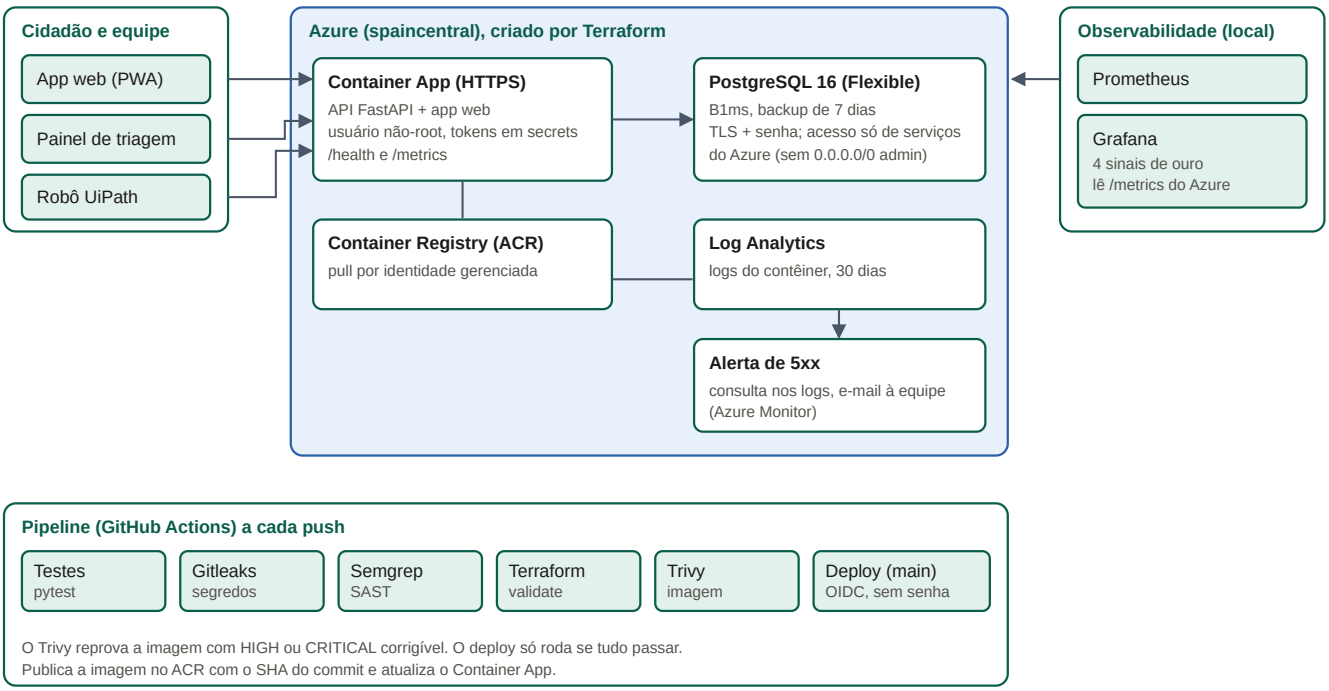
Secure DevOps Tools & Cloud Computing · Prof. Cleórbete Santos · GS 2026.2 CitizenTech

Zela: Zeladoria Urbana por Foto. O cidadão envia uma foto e um local, confirma a categoria que a IA sugere e recebe um protocolo para acompanhar o relato.

Integrantes	Pedro Henrique de Assunção Lima (RM552746) · Joseh Gabriel Trimboli Agra (RM553094)
MVP em URL pública	https://ca-zela-uylmp.jollyglacier-50c67627.spaincentral.azurecontainerapps.io
Repositório público	https://github.com/phz1n777/zela
Dockerfile	api/Dockerfile · docker-compose.yml
Terraform	infra/terraform/
Workflow	.github/workflows/ci.yml
Observabilidade	painel do Grafana · regras do Prometheus
Documentos	arquitetura · custos e LGPD · evidências · ficha técnica

> **Transparência sobre o pipeline.** Os runners hospedados do GitHub Actions estão bloqueados na conta do repositório por um problema de cobrança do GitHub ("account is locked due to a billing issue"). O pipeline foi executado num **runner próprio**, registrado no repositório e rodando no computador do desenvolvedor, e depois removido. Na execução 43093596676 (commit 52c4321) os 6 jobs terminaram em sucesso, inclusive o deploy. A seção 9 lista as limitações conhecidas.

1. Arquitetura



Código-fonte do diagrama em docs/arquitetura.md. Toda a infraestrutura do Azure é criada por Terraform. O pipeline só publica a imagem.

Fluxo de entrega

1. Um push na `main` dispara o workflow `ci`.

2. Rodam em paralelo: testes (pytest), Gitleaks (segredos no histórico), Semgrep (SAST de Python e Dockerfile) e `terraform validate`.
3. O Trivy varre a imagem e reprova o pipeline se houver vulnerabilidade HIGH ou CRITICAL com correção disponível.
4. Só se tudo passar, o job `deploy` entra no Azure por OIDC (sem senha guardada), publica a imagem no ACR com o SHA do commit, atualiza o Container App e confere o `/health`.

2. Container e infraestrutura como código

Critério	Como foi atendido
Dockerfile enxuto, usuário não root	Base `python:3.12-slim`, `--no-cache-dir`, usuário `zela` (uid 10001) e `HEALTHCHECK`. O YOLO é opcional (`COM_VISAO=1`) para a imagem não crescer ~1 GB sem necessidade.
Sem segredo na imagem	Tokens, senha do banco e `DATABASE_URL` chegam por variáveis de ambiente (*secrets* do Container App). O `.env` é ignorado pelo git e o Gitleaks varre o histórico.
Stack sobe com docker compose	`docker compose up` sobe API e Postgres. `docker-compose.monitoring.yml` adiciona Prometheus e Grafana.
Nuvem criada por Terraform	14 recursos em `infra/terraform/`: resource group, ACR, Postgres, Log Analytics, ambiente e Container App, identidade gerenciada, alerta e grupo de ação. O `subscription_id` fica fixado no provider.
State e .tfvars fora do repositório	`*.tfstate`, `*.tfvars`, `*.plan` e `.terraform/` estão no `.gitignore`. Só o `terraform.tfvars.example` vai ao git.
Porta administrativa nunca aberta para 0.0.0.0/0	Só a API é pública (HTTPS). O banco aceita apenas a regra "serviços do Azure" (início e fim 0.0.0.0, que não é a internet), com TLS e senha. Nenhuma porta de administração (SSH, 5432 aberta ao mundo) é exposta.

3. Pipeline DevSecOps

Workflow: .github/workflows/ci.yml, com 6 jobs: `testes`, `gitleaks`, `semgrep`, `terraform`, `imagem` (Trivy) e `deploy`. Roda a cada push e pull request, e o `deploy` só na `main`, depois de todos os outros. O runner é escolhido pela variável opcional `RUNNER_LABEL` (sem ela, usa o runner do GitHub). O login no Azure é por OIDC, com os identificadores em GitHub Secrets (`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`) e sem senha guardada.

Resultado no GitHub Actions

Execução 43093596676, commit `52c4321`, em runner próprio:

Job	Ferramenta	Resultado
testes	pytest (dentro de `api/`)	**sucesso (30 testes localmente)**
gitleaks	Gitleaks 8.30.1, histórico git inteiro	**sucesso, nenhum segredo**
semgrep	Semgrep 1.97.0, regras `p/python` e `p/dockerfile`	**sucesso (158 regras, 0 achados localmente)**
terraform	`terraform validate` e `fmt`	**sucesso**
imagem	Trivy 0.76.2, HIGH e CRITICAL com correção	**sucesso (0 vulnerabilidades)**
deploy	OIDC, build, push no ACR, atualização do Container App, teste de fumaça	**sucesso: imagem `zela-api:52c4321e5445` no ar**

Até chegar aqui foram três execuções com falha, todas por defeitos do próprio workflow e não por achados de segurança: o `pytest` rodava fora de `api/`; a action do Gitleaks quebrava no envio do relatório e varria só 1 commit; a versão da action do Trivy não existia; e a credencial OIDC usava o formato antigo do *subject* (o GitHub agora inclui os IDs numéricos do dono e do repositório). Foram corrigidos com Gitleaks e Trivy por Docker e uma segunda credencial federada. Os testes, o Semgrep e o `terraform validate` também foram rodados localmente em 11/11/2026.

Uma execução barrada e a corrigida

A primeira varredura do Trivy **reprovou** a imagem (commit 0855ed2) com 7 achados HIGH corrigíveis:

Origem	Pacote	Correção
Sistema (Debian 13)	`apt` e `libapt-pkg7.0` (CVE-2026-109518)	`apt-get upgrade` no Dockerfile (apt 3.0.4)
Sistema (Debian 13)	`zlib1g` (CVE-2026-76844, CVE-2026-85091)	zlib `1.3.dfsg+really1.3.1-1+deb13u1`
Python	`starlette` 0.46.2 (CVE-2025-62727, CVE-2026-48818, CVE-2026-54283)	FastAPI 0.162 e Starlette 1.3.1 ou superior (instalado: 1.10)

O commit c8fc16d corrigiu os 7 achados, e o Trivy passou a sair com código 0. Essa reprovação e a correção foram verificadas **localmente**, antes de o pipeline rodar no GitHub, então a evidência de "barrada e corrigida" não é uma execução do Actions. Saída das ferramentas em infra/evidencias/.

4. Observabilidade

O Prometheus lê o `/metrics` do MVP publicado no Azure e o Grafana mostra o painel **Zela: 4 sinais de ouro**. A imagem abaixo é do painel lendo a API em produção, com tráfego e erros provocados de propósito.

Sinal	Consulta
Latência	`histogram_quantile(0.95, sum(rate(zela_latencia_segundos_bucket[5m])) by (le))` (também p50)
Tráfego	`sum(rate(zela_requisicoes_total[5m])) by (rota)`
Erros	% de respostas `status=~"5.."` sobre o total
Saturação	`rate(process_cpu_seconds_total[5m])` e `process_resident_memory_bytes`



Alerta disparado

O alerta `zela-erros-5xx` (Azure Monitor, consulta nos logs do contêiner) avisa por e-mail quando passa de 5 respostas 5xx em 5 minutos. Depois de 10 respostas 500 provocadas de propósito, ele entrou em **Fired** às **19:36 UTC de 11/11/2026** (gravidade Sev2). A prova está em infra/evidencias/alerta_disparado.json. Não foi verificado se o e-mail chegou à caixa de entrada.

Dois problemas foram achados e corrigidos no caminho: a métrica `Requests` do Container App não entrega a divisão por categoria de status, então o primeiro alerta (por métrica) nunca dispararia e foi trocado por consulta nos logs; e erros 500 não tratados não eram contados no `/metrics` da API, o que foi corrigido com teste automático.

5. Responsabilidade compartilhada

Camada	Quem responde	O que a equipe faz
Datacenter, hardware e rede física	Microsoft	Escolher a região permitida pela assinatura (`spaincentral`)
Serviços gerenciados (Container Apps, Postgres, ACR, Log Analytics)	Microsoft: disponibilidade e patches do serviço	Configurar tamanho, backup de 7 dias e retenção de logs de 30 dias
Imagem e dependências	**Equipe**	Manter sem CVE alta (Trivy), usuário não-root, sem segredo na imagem
Código da aplicação	**Equipe**	Validar entradas, limitar uploads, remover EXIF, proteger rotas de triagem e do robô por token
Identidades e segredos	**Equipe**	Secrets do Container App e do GitHub, identidade gerenciada para o pull do ACR, OIDC no deploy, state e tfvars fora do git
Dados pessoais (LGPD)	**Equipe** (controladora no piloto)	Minimizar, documentar e responder ao titular
Monitoramento e resposta	**Equipe**	Painel, alerta por e-mail e a decisão de agir

6. Dados pessoais (LGPD)

Dados tratados: **foto** da via, **localização** (texto e coordenadas) e **descrição livre**. Ficam no PostgreSQL e no disco do contêiner, na região `spaincentral` (União Europeia, o que configura transferência internacional). O relato é **anônimo**: não coletamos nome, CPF, telefone nem e-mail. A foto tem o EXIF (GPS e modelo do aparelho) removido no upload. O OpenRouter recebe só as dúvidas de uso do FonteBot, nunca fotos ou relatos. Detalhes em docs/custos_e_lgpd.md.

Riscos conhecidos que ainda não foram tratados

- **Leitura pública da API:** a listagem de relatos, o relato por id e a foto não exigem login, porque o painel da demo as consome. Qualquer pessoa com a URL vê localizações e fotos.
- **Sem rota de exclusão nem prazo de retenção** definido para os relatos.
- **Fotos em disco efêmero:** somem a cada novo deploy do contêiner. Azure Files resolveria.
- **Texto livre** pode receber dados pessoais. **/metrics** é público (só contadores agregados).
- **Estado do Terraform local:** guarda a senha do banco. Em produção, usar backend remoto criptografado.

7. Custos

Recurso	Configuração
Container App	0,5 vCPU e 1 GiB, 1 a 2 réplicas, HTTPS público
PostgreSQL Flexible Server	Burstable B1ms, 32 GB, backup de 7 dias
Container Registry	Basic
Log Analytics e alerta	Retenção de 30 dias, 1 alerta e 1 grupo de ação

Estimativa: entre US$ 30 e US$ 60 por mês com o piloto ligado o tempo todo. É uma estimativa por preços públicos, não um valor medido; o saldo real está em Cost Management no portal do Azure. O crédito do Azure for Students (cerca de US$ 100) cobre o período. O ambiente fica no ar durante a correção (cerca de uma

semana depois de 19/11) e só então é destruído com `terraform destroy`. Mais formas de reduzir o custo em docs/custos_e_lgpd.md.

8. Como reproduzir

```
git clone https://github.com/phz1n777/zela.git && cd zela
cp .env.example .env            # preencha tokens e senha
docker compose up --build       # API em http://localhost:8000

# Monitoramento contra o MVP publicado
PROM_CONFIG=prometheus-azure.yml docker compose -f docker-compose.monitoring.yml up prometheus
grafana

# Nuvem (conta Azure for Students)
cd infra/terraform && cp terraform.tfvars.example terraform.tfvars
terraform init && terraform plan -out tf.plan && terraform apply tf.plan
```

README completo em infra/README.md, com o passo a passo do deploy e da configuração do acesso OIDC.

9. Limitações conhecidas

Pipeline e entrega

- **Sem runner hospedado do GitHub.** A cobrança da conta está bloqueada. A execução verde foi feita num runner no computador do desenvolvedor, que depois foi removido (um runner próprio em repositório público é um risco: um pull request malicioso executaria código na máquina). Por isso, **novos pushes não rodam o pipeline** enquanto o bloqueio durar, e a evidência do pipeline é essa execução e as saídas locais.
- **"Barrada e corrigida" não é uma execução do Actions.** A reprovação do Trivy e a correção foram verificadas localmente.
- **O Terraform não é aplicado pelo pipeline.** O CI só roda `validate` e `fmt`; o `apply` é manual, e o state fica na máquina do desenvolvedor (com a senha do banco).
- **Escopo das verificações.** O `test_vision.py` fica fora do CI, o Trivy varre só vulnerabilidades (não configuração) e o Semgrep usa regras da comunidade. O teste de fumaça do deploy confere apenas o `/health`.
- **O deploy automático foi validado em uma execução.** O acesso OIDC tem papel Contributor restrito ao grupo de recursos do Zela.

Observabilidade

- **Prometheus e Grafana rodam localmente**, apontando para o MVP no Azure. Não há dashboard hospedado nem Application Insights, e as regras do Prometheus não notificam ninguém (não há Alertmanager). Quem avisa a equipe é o alerta do Azure Monitor.
- **O e-mail do alerta não foi confirmado.** O alerta entrou em `Fired`, mas não foi verificado se a mensagem chegou à caixa de entrada.
- O Grafana local usa a senha padrão `admin`, adequada só para uso local.

Produto, dados e infraestrutura

- **Sem sugestão de dano pela IA em produção.** A imagem publicada roda sem o YOLO (o modelo ainda não foi treinado), então o relato não recebe a sugestão automática.
- **Leitura pública da API** (listagem, relato e foto), **sem rota de exclusão nem prazo de retenção**, e texto livre sem filtro de dados pessoais. Detalhes na seção 6.
- **Fotos em disco efêmero:** somem a cada novo deploy do contêiner.

- **Banco sem rede privada:** aceita apenas a regra "serviços do Azure", com TLS e senha. Para produção, o certo é rede virtual e endpoint privado. O `/metrics` também é público.
- **Região imposta pela assinatura:** `spaincentral` é uma das poucas permitidas, o que implica transferência internacional de dados.
- **Custo estimado, não medido** (US$ 30 a 60 por mês). O ambiente precisa ficar no ar durante a correção, e o crédito de estudante limita o orçamento.
- Instância única e sem alta disponibilidade, adequada a um piloto.